

Investment Portfolio Performance and Risk Analytics

Developed by: Eric Rathod

Project Overview

This project analyzes the historical performance and risk of a diversified investment portfolio using Python, SQL, and Power BI.

The project collects historical market data, calculates key investment performance metrics, stores processed results in a SQLite database, and prepares datasets for an interactive Power BI dashboard.

Business Objectives

- Evaluate portfolio and individual asset performance
- Measure portfolio risk and volatility
- Compare portfolio performance against the S&P 500 benchmark
- Identify the strongest and weakest performing investments
- Automate the preparation of investment reporting data

Tools Used

- Python
- Pandas
- NumPy
- Yahoo Finance
- SQLite
- Matplotlib
- Plotly
- Power BI

```
1 # Install the libraries required for this project
2 !pip install -q yfinance plotly
3
```

```
1 # Data collection and manipulation
2 import yfinance as yf
3 import pandas as pd
4 import numpy as np
5
6 # Data visualization
7 import matplotlib.pyplot as plt
8 import plotly.express as px
9 import plotly.graph_objects as go
10
11 # Database and file management
12 import sqlite3
13 import os
14 import warnings
15
16 # Display settings
17 warnings.filterwarnings("ignore")
18 pd.set_option("display.max_columns", None)
19 pd.set_option("display.float_format", lambda x: f"{x:,.4f}")
20
21 print("Libraries imported successfully.")
```

Libraries imported successfully.

```
1 # Create folders for project outputs
2
3 folders = [
4     "data",
5     "database",
6     "charts"
7 ]
8
9 for folder in folders:
10     os.makedirs(folder, exist_ok=True)
```

Project folders created successfully.

```

1 # Define the assets included in the portfolio
2
3 portfolio = {
4     "AAPL": {
5         "company": "Apple",
6         "sector": "Technology",
7         "weight": 0.15
8     },
9     "MSFT": {
10        "company": "Microsoft",
11        "sector": "Technology",
12        "weight": 0.15
13    },
14    "NVDA": {
15        "company": "NVIDIA",
16        "sector": "Technology",
17        "weight": 0.10
18    },
19    "AMZN": {
20        "company": "Amazon",
21        "sector": "Consumer Discretionary",
22        "weight": 0.10
23    },
24    "JPM": {
25        "company": "JPMorgan Chase",
26        "sector": "Financial Services",
27        "weight": 0.15
28    },
29    "SPY": {
30        "company": "S&P 500 ETF",
31        "sector": "Broad Market",
32        "weight": 0.15
33    },
34    "GLD": {
35        "company": "Gold ETF",
36        "sector": "Commodities",
37        "weight": 0.10
38    },
39    "BND": {
40        "company": "Total Bond Market ETF",
41        "sector": "Fixed Income",
42        "weight": 0.10
43    }
44 }
45
46 initial_investment = 100000
47
48 print("Number of assets:", len(portfolio))
49 print("Total portfolio weight:", sum(asset["weight"] for asset in portfolio.values()))
50 print("Initial investment: $", f"{initial_investment:,.2f}")

```

Number of assets: 8
Total portfolio weight: 1.0
Initial investment: \$ 100,000.00

```

1 # Convert the portfolio dictionary into a DataFrame
2
3 portfolio_df = pd.DataFrame.from_dict(portfolio, orient="index")
4
5 portfolio_df.index.name = "Ticker"
6 portfolio_df = portfolio_df.reset_index()
7
8 portfolio_df["Initial_Investment"] = (
9     portfolio_df["weight"] * initial_investment
10 )
11
12 portfolio_df

```

	Ticker	company	sector	weight	Initial_Investment	
0	AAPL	Apple	Technology	0.1500	15,000.0000	
1	MSFT	Microsoft	Technology	0.1500	15,000.0000	
2	NVDA	NVIDIA	Technology	0.1000	10,000.0000	
3	AMZN	Amazon	Consumer Discretionary	0.1000	10,000.0000	
4	JPM	JPMorgan Chase	Financial Services	0.1500	15,000.0000	
5	SPY	S&P 500 ETF	Broad Market	0.1500	15,000.0000	
6	GLD	Gold ETF	Commodities	0.1000	10,000.0000	
7	BND	Total Bond Market ETF	Fixed Income	0.1000	10,000.0000	

Next steps:

[Generate code with portfolio_df](#)[New interactive sheet](#)

```

1 # Define the historical analysis period
2
3 start_date = "2023-01-01"
4 end_date = "2026-07-20"
5
6 tickers = list(portfolio.keys())
7
8 print("Assets:", tickers)
9 print("Analysis period:", start_date, "to", end_date)

```

Assets: ['AAPL', 'MSFT', 'NVDA', 'AMZN', 'JPM', 'SPY', 'GLD', 'BND']
 Analysis period: 2023-01-01 to 2026-07-20

```

1 # Download adjusted historical closing prices
2
3 price_data = yf.download(
4     tickers,
5     start=start_date,
6     end=end_date,
7     auto_adjust=True,
8     progress=False
9 )
10
11 print("Market data downloaded successfully.")
12 print("Rows:", price_data.shape[0])
13 print("Columns:", price_data.shape[1])

```

Market data downloaded successfully.
 Rows: 887
 Columns: 40

```

1 # Extract adjusted closing prices
2
3 if isinstance(price_data.columns, pd.MultiIndex):
4     closing_prices = price_data["Close"].copy()
5 else:
6     closing_prices = price_data.copy()
7
8 closing_prices = closing_prices[tickers]
9
10 closing_prices.head()

```

Ticker	AAPL	MSFT	NVDA	AMZN	JPM	SPY	GLD	BND	
Date									
2023-01-03	122.9827	232.9483	14.2833	85.8200	123.7352	364.1340	171.0600	63.6491	
2023-01-04	124.2512	222.7584	14.7163	85.1400	124.8890	366.9452	172.6700	64.0104	
2023-01-05	122.9336	216.1563	14.2334	83.1200	124.8614	362.7570	170.5200	63.9398	
2023-01-06	127.4568	218.7038	14.8261	86.0800	127.2506	371.0759	173.7100	64.6449	
2023-01-09	127.9779	220.8331	15.5934	87.3600	126.7248	370.8655	174.1000	64.8212	

Next steps:

[Generate code with closing_prices](#)[New interactive sheet](#)

```

1 # Check dataset structure and missing values
2
3 print("Dataset shape:", closing_prices.shape)

```

```

4
5 print("\nMissing values by asset:")
6 print(closing_prices.isnull().sum())
7
8 print("\nFirst available date:")
9 print(closing_prices.index.min())
10
11 print("\nLast available date:")
12 print(closing_prices.index.max())

```

Dataset shape: (887, 8)

Missing values by asset:

```

Ticker
AAPL    0
MSFT    0
NVDA    0
AMZN    0
JPM     0
SPY     0
GLD     0
BND     0
dtype: int64

```

First available date:
2023-01-03 00:00:00

Last available date:
2026-07-17 00:00:00

```

1 # Clean missing market-price values
2
3 closing_prices = closing_prices.sort_index()
4
5 # Fill small gaps using the previous available trading-day price
6 closing_prices = closing_prices.ffill()
7
8 # Remove any remaining rows with missing values
9 closing_prices = closing_prices.dropna()
10
11 print("Cleaned dataset shape:", closing_prices.shape)
12 print("Remaining missing values:", closing_prices.isnull().sum().sum())

```

Cleaned dataset shape: (887, 8)
Remaining missing values: 0

```

1 # Save the clean historical-price dataset
2
3 closing_prices.to_csv(
4     "data/portfolio_prices.csv",
5     index=True
6 )
7
8 print("portfolio_prices.csv saved successfully.")

```

portfolio_prices.csv saved successfully.

```

1 # Display summary statistics for historical asset prices
2
3 price_summary = closing_prices.describe().T
4
5 price_summary = price_summary.rename(
6     columns={
7         "count": "Trading_Days",
8         "mean": "Average_Price",
9         "std": "Price_Standard_Deviation",
10        "min": "Minimum_Price",
11        "max": "Maximum_Price"
12    }
13 )
14
15 price_summary[
16     [
17         "Trading_Days",
18         "Average_Price",
19         "Price_Standard_Deviation",
20         "Minimum_Price",
21         "Maximum_Price"

```

```
22 ]
23 # Save data to CSV
```

	Trading_Days	Average_Price	Price_Standard_Deviation	Minimum_Price	Maximum_Price	
Ticker						
AAPL	887.0000	213.6900	42.3400	122.9300	333.7400	
MSFT	887.0000	396.0900	70.4600	216.1600	538.6600	
NVDA	887.0000	113.9000	60.6300	14.2300	235.4700	
AMZN	887.0000	184.0300	46.7900	83.1200	274.9900	
JPM	887.0000	217.7700	68.1300	115.2300	346.9100	
SPY	887.0000	545.9300	107.9400	362.7600	757.6200	
GLD	887.0000	268.0900	90.1500	168.3500	495.9000	
BND	887.0000	68.2500	3.2900	61.3500	73.9400	

```
1 # Calculate daily percentage returns for each asset
2
3 daily_returns = closing_prices.pct_change().dropna()
4
5 print("Daily returns calculated successfully.")
6 print("Dataset shape:", daily_returns.shape)
7
8 daily_returns.head()
```

Daily returns calculated successfully.
Dataset shape: (886, 8)

Ticker	AAPL	MSFT	NVDA	AMZN	JPM	SPY	GLD	BND	
Date									
2023-01-04	0.0103	-0.0437	0.0303	-0.0079	0.0093	0.0077	0.0094	0.0057	
2023-01-05	-0.0106	-0.0296	-0.0328	-0.0237	-0.0002	-0.0114	-0.0125	-0.0011	
2023-01-06	0.0368	0.0118	0.0416	0.0356	0.0191	0.0229	0.0187	0.0110	
2023-01-09	0.0041	0.0097	0.0518	0.0149	-0.0041	-0.0006	0.0022	0.0027	
2023-01-10	0.0045	0.0076	0.0180	0.0287	0.0090	0.0070	0.0037	-0.0039	

Next steps: [Generate code with daily_returns](#) [New interactive sheet](#)

```
1 # Save daily returns for later SQL and Power BI use
2
3 daily_returns.to_csv(
4     "data/daily_returns.csv",
5     index=True
6 )
7
8 print("daily_returns.csv saved successfully.")
```

daily_returns.csv saved successfully.

```
1 # Create a weight series in the same ticker order as the price data
2
3 weights = pd.Series(
4     {ticker: portfolio[ticker]["weight"] for ticker in tickers}
5 )
6
7 weights = weights[closing_prices.columns]
8
9 print("Portfolio weights:")
10 print(weights)
11
12 print("\nTotal weight:", weights.sum())
```

Portfolio weights:

Ticker	
AAPL	0.1500
MSFT	0.1500
NVDA	0.1000
AMZN	0.1000
JPM	0.1500
SPY	0.1500

```
GLD    0.1000
BND    0.1000
dtype: float64
```

Total weight: 1.0

```
1 # Calculate the weighted daily return of the full portfolio
2
3 portfolio_daily_return = daily_returns.dot(weights)
4
5 portfolio_returns_df = pd.DataFrame({
6     "Portfolio_Daily_Return": portfolio_daily_return
7 })
8
9 portfolio_returns_df.head()
```

Portfolio_Daily_Return 	
Date	
2023-01-04	0.0013
2023-01-05	-0.0148
2023-01-06	0.0243
2023-01-09	0.0085
2023-01-10	0.0089

Next steps: [Generate code with portfolio_returns_df](#) [New interactive sheet](#)

```
1 # Calculate cumulative portfolio performance
2
3 portfolio_returns_df["Portfolio_Cumulative_Return"] = (
4     1 + portfolio_returns_df["Portfolio_Daily_Return"]
5 ).cumprod() - 1
6
7 portfolio_returns_df["Portfolio_Value"] = (
8     initial_investment
9     * (1 + portfolio_returns_df["Portfolio_Cumulative_Return"])
10 )
11
12 portfolio_returns_df.tail()
```

Portfolio_Daily_Return Portfolio_Cumulative_Return Portfolio_Value 			
Date			
2026-07-13	-0.0045	1.7554	275,543.4749
2026-07-14	0.0066	1.7736	277,355.8317
2026-07-15	0.0161	1.8182	281,824.6059
2026-07-16	-0.0041	1.8067	280,665.3253
2026-07-17	-0.0072	1.7866	278,658.3589

```
1 # Calculate the latest portfolio value and total profit or loss
2
3 current_portfolio_value = portfolio_returns_df["Portfolio_Value"].iloc[-1]
4
5 total_profit_loss = current_portfolio_value - initial_investment
6
7 total_return_percent = (
8     total_profit_loss / initial_investment
9 ) * 100
10
11 print("Initial investment:", f"${initial_investment:,.2f}")
12 print("Current portfolio value:", f"${current_portfolio_value:,.2f}")
13 print("Total profit/loss:", f"${total_profit_loss:,.2f}")
14 print("Total return:", f"${total_return_percent:,.2f}%")
```

Initial investment: \$100,000.00
Current portfolio value: \$278,658.36

```
Total profit/loss: $178,658.36
Total return: 178.66%
```

```
1 # Calculate annualized portfolio return
2
3 trading_days = 252
4
5 average_daily_return = portfolio_daily_return.mean()
6
7 annualized_return = (
8     (1 + average_daily_return) ** trading_days
9 ) - 1
10
11 print("Annualized return:", f"{annualized_return * 100:.2f}%")
```

```
Annualized return: 35.54%
```

```
1 # Calculate annualized portfolio volatility
2
3 annualized_volatility = (
4     portfolio_daily_return.std()
5     * np.sqrt(trading_days)
6 )
7
8 print("Annualized volatility:", f"{annualized_volatility * 100:.2f}%")
```

```
Annualized volatility: 15.95%
```

```
1 # Calculate the portfolio Sharpe ratio
2
3 risk_free_rate = 0.04
4
5 sharpe_ratio = (
6     annualized_return - risk_free_rate
7 ) / annualized_volatility
8
9 print("Sharpe ratio:", round(sharpe_ratio, 2))
```

```
Sharpe ratio: 1.98
```

```
1 # Calculate portfolio drawdown
2
3 portfolio_returns_df["Running_Peak"] = (
4     portfolio_returns_df["Portfolio_Value"].cummax()
5 )
6
7 portfolio_returns_df["Drawdown"] = (
8     portfolio_returns_df["Portfolio_Value"]
9     / portfolio_returns_df["Running_Peak"]
10 ) - 1
11
12 maximum_drawdown = portfolio_returns_df["Drawdown"].min()
13
14 print("Maximum drawdown:", f"{maximum_drawdown * 100:.2f}%")
```

```
Maximum drawdown: -18.33%
```

```
1 # Save portfolio-level daily performance
2
3 portfolio_returns_df.to_csv(
4     "data/portfolio_performance.csv",
5     index=True
6 )
7
8 print("portfolio_performance.csv saved successfully.")
```

```
portfolio_performance.csv saved successfully.
```

```
1 # Create a summary table of portfolio KPIs
2
3 portfolio_kpi_summary = pd.DataFrame({
4     "Metric": [
5         "Initial Investment",
6         "Current Portfolio Value",
7         "Total Profit or Loss",
```

```

8     "Total Return Percent",
9     "Annualized Return Percent",
10    "Annualized Volatility Percent",
11    "Sharpe Ratio",
12    "Maximum Drawdown Percent"
13 ],
14 "Value": [
15     initial_investment,
16     current_portfolio_value,
17     total_profit_loss,
18     total_return_percent,
19     annualized_return * 100,
20     annualized_volatility * 100,
21     sharpe_ratio,
22     maximum_drawdown * 100
23 ]
24 })
25
26 portfolio_kpi_summary

```

	Metric	Value	
0	Initial Investment	100,000.0000	
1	Current Portfolio Value	278,658.3589	
2	Total Profit or Loss	178,658.3589	
3	Total Return Percent	178.6584	
4	Annualized Return Percent	35.5422	
5	Annualized Volatility Percent	15.9506	
6	Sharpe Ratio	1.9775	
7	Maximum Drawdown Percent	-18.3253	

Next steps:

[Generate code with portfolio_kpi_summary](#)[New interactive sheet](#)

```

1 # Save portfolio KPI summary
2
3 portfolio_kpi_summary.to_csv(
4     "data/portfolio_kpi_summary.csv",
5     index=False
6 )
7
8 print("portfolio_kpi_summary.csv saved successfully.")

```

portfolio_kpi_summary.csv saved successfully.

```

1 import yfinance as yf
2 import pandas as pd
3 import numpy as np

```

```

1 portfolio = {
2     "AAPL": {"company": "Apple", "sector": "Technology", "weight": 0.15},
3     "MSFT": {"company": "Microsoft", "sector": "Technology", "weight": 0.15},
4     "NVDA": {"company": "NVIDIA", "sector": "Technology", "weight": 0.10},
5     "AMZN": {"company": "Amazon", "sector": "Consumer Discretionary", "weight": 0.10},
6     "JPM": {"company": "JPMorgan Chase", "sector": "Financial Services", "weight": 0.15},
7     "SPY": {"company": "S&P 500 ETF", "sector": "Broad Market", "weight": 0.15},
8     "GLD": {"company": "Gold ETF", "sector": "Commodities", "weight": 0.10},
9     "BND": {"company": "Total Bond Market ETF", "sector": "Fixed Income", "weight": 0.10}
10 }
11
12 tickers = list(portfolio.keys())

```

```

1 price_data = yf.download(
2     tickers,
3     start="2023-01-01",

```



```

4     end="2026-07-20",
5     auto_adjust=True,
6     progress=False
7 )

```

```

1 closing_prices = price_data["Close"]
2
3 closing_prices.head()

```

Ticker	AAPL	AMZN	BND	GLD	JPM	MSFT	NVDA	SPY	
Date									
2023-01-03	122.982712	85.820000	63.649029	171.059998	123.735191	232.948257	14.283263	364.133942	
2023-01-04	124.251183	85.139999	64.010330	172.669998	124.889030	222.758377	14.716301	366.945129	
2023-01-05	122.933556	83.120003	63.939835	170.520004	124.861351	216.156311	14.233374	362.757050	
2023-01-06	127.456772	86.080002	64.644867	173.710007	127.250671	218.703781	14.826057	371.075867	
2023-01-09	127.977943	87.360001	64.821190	174.100006	126.724815	220.833191	15.593351	370.865448	

Next steps: [Generate code with closing_prices](#) [New interactive sheet](#)

```

1 # Calculate total return for each asset
2
3 asset_total_returns = (
4     closing_prices.iloc[-1] / closing_prices.iloc[0]
5 ) - 1
6
7 asset_total_returns

```

	0
Ticker	
AAPL	1.713715
AMZN	1.880797
BND	0.144715
GLD	1.153689
JPM	1.756694
MSFT	0.690590
NVDA	13.199136
SPY	1.041254

dtype: float64

```

1 # Calculate daily returns
2
3 daily_returns = closing_prices.pct_change().dropna()
4
5 print("Daily returns created successfully.")
6 daily_returns.head()

```

Daily returns created successfully.

Ticker	AAPL	AMZN	BND	GLD	JPM	MSFT	NVDA	SPY	
Date									
2023-01-04	0.010314	-0.007924	0.005676	0.009412	0.009325	-0.043743	0.030318	0.007720	
2023-01-05	-0.010605	-0.023726	-0.001101	-0.012451	-0.000222	-0.029638	-0.032816	-0.011413	
2023-01-06	0.036794	0.035611	0.011026	0.018707	0.019136	0.011785	0.041640	0.022932	
2023-01-09	0.004089	0.014870	0.002728	0.002245	-0.004132	0.009737	0.051753	-0.000567	
2023-01-10	0.004456	0.028732	-0.003943	0.003676	0.008954	0.007617	0.017981	0.007013	

Next steps: [Generate code with daily_returns](#) [New interactive sheet](#)

```

1 # Step 29: Calculate annualized return for each asset
2
3 trading_days = 252
4

```

```

5 asset_annualized_returns = (
6     (1 + daily_returns.mean()) ** trading_days
7 ) - 1
8
9 asset_annualized_returns

```

0

Ticker	
AAPL	0.372668
AMZN	0.420732
BND	0.040819
GLD	0.268671
JPM	0.370707
MSFT	0.198028
NVDA	1.388129
SPY	0.239012

dtype: float64

```

1 # Step 30: Calculate annualized volatility for each asset
2
3 asset_annualized_volatility = (
4     daily_returns.std() * np.sqrt(trading_days)
5 )
6
7 asset_annualized_volatility

```

0

Ticker	
AAPL	0.257307
AMZN	0.317804
BND	0.056020
GLD	0.198347
JPM	0.232372
MSFT	0.250637
NVDA	0.486104
SPY	0.151250

dtype: float64

```

1 # Step 31: Calculate Sharpe ratio for each asset
2
3 risk_free_rate = 0.04
4
5 asset_sharpe_ratios = (
6     asset_annualized_returns - risk_free_rate
7 ) / asset_annualized_volatility
8
9 asset_sharpe_ratios

```

```

0

Ticker
AAPL  1.292882
AMZN  1.198008
BND   0.014627
GLD   1.152885
JPM   1.423177
MSFT  0.630505
NVDA  2.773332
SPY   1.315784

dtype: float64

```

```

1 # Step 32: Calculate maximum drawdown for each asset
2
3 asset_cumulative_growth = (1 + daily_returns).cumprod()
4
5 asset_running_peak = asset_cumulative_growth.cummax()
6
7 asset_drawdowns = (
8     asset_cumulative_growth / asset_running_peak
9 ) - 1
10
11 asset_max_drawdown = asset_drawdowns.min()
12
13 asset_max_drawdown

```

```

0

Ticker
AAPL  -0.333605
AMZN  -0.308766
BND   -0.071770
GLD   -0.264045
JPM   -0.244239
MSFT  -0.344984
NVDA  -0.368810
SPY   -0.187552

dtype: float64

```

```

1 # Step 33: Create the portfolio information table
2
3 portfolio = {
4     "AAPL": {"Company": "Apple", "Sector": "Technology", "Weight": 0.15},
5     "MSFT": {"Company": "Microsoft", "Sector": "Technology", "Weight": 0.15},
6     "NVDA": {"Company": "NVIDIA", "Sector": "Technology", "Weight": 0.10},
7     "AMZN": {"Company": "Amazon", "Sector": "Consumer Discretionary", "Weight": 0.10},
8     "JPM": {"Company": "JPMorgan Chase", "Sector": "Financial Services", "Weight": 0.15},
9     "SPY": {"Company": "SPDR S&P 500 ETF", "Sector": "ETF", "Weight": 0.15},
10    "GLD": {"Company": "SPDR Gold Shares", "Sector": "Commodity", "Weight": 0.10},
11    "BND": {"Company": "Vanguard Total Bond Market ETF", "Sector": "Bond ETF", "Weight": 0.10}
12 }
13
14 initial_investment = 100000
15
16 portfolio_df = pd.DataFrame.from_dict(
17     portfolio,
18     orient="index"
19 )
20
21 portfolio_df.index.name = "Ticker"
22
23 portfolio_df = portfolio_df.reset_index()
24
25 portfolio_df["Initial_Investment"] = (
26     portfolio_df["Weight"] * initial_investment

```

```

27 )
28
29 portfolio_df

```

	Ticker	Company	Sector	Weight	Initial_Investment	
0	AAPL	Apple	Technology	0.15	15000.0	 
1	MSFT	Microsoft	Technology	0.15	15000.0	
2	NVDA	NVIDIA	Technology	0.10	10000.0	
3	AMZN	Amazon	Consumer Discretionary	0.10	10000.0	
4	JPM	JPMorgan Chase	Financial Services	0.15	15000.0	
5	SPY	SPDR S&P 500 ETF	ETF	0.15	15000.0	
6	GLD	SPDR Gold Shares	Commodity	0.10	10000.0	
7	BND	Vanguard Total Bond Market ETF	Bond ETF	0.10	10000.0	

Next steps: [Generate code with portfolio_df](#) [New interactive sheet](#)

```

1 # Step 34: Create the final asset summary table
2
3 asset_summary = portfolio_df.copy()
4
5 asset_summary["Starting_Price"] = asset_summary["Ticker"].map(
6     closing_prices.iloc[0]
7 )
8
9 asset_summary["Latest_Price"] = asset_summary["Ticker"].map(
10     closing_prices.iloc[-1]
11 )
12
13 asset_summary["Total_Return_Percent"] = asset_summary["Ticker"].map(
14     asset_total_returns * 100
15 )
16
17 asset_summary["Annualized_Return_Percent"] = asset_summary["Ticker"].map(
18     asset_annualized_returns * 100
19 )
20
21 asset_summary["Annualized_Volatility_Percent"] = asset_summary["Ticker"].map(
22     asset_annualized_volatility * 100
23 )
24
25 asset_summary["Sharpe_Ratio"] = asset_summary["Ticker"].map(
26     asset_sharpe_ratios
27 )
28
29 asset_summary["Maximum_Drawdown_Percent"] = asset_summary["Ticker"].map(
30     asset_max_drawdown * 100
31 )
32
33 asset_summary["Current_Value"] = asset_summary["Ticker"].map(
34     initial_investment * asset_summary["Weight"] * (1 + asset_total_returns)
35 )
36
37 asset_summary["Profit_Loss"] = (
38     asset_summary["Current_Value"]
39     - asset_summary["Initial_Investment"]
40 )
41
42 asset_summary.round(2)

```

	Ticker	Company	Sector	Weight	Initial_Investment	Starting_Price	Latest_Price	Total_Return_Percent	Annualized
0	AAPL	Apple	Technology	0.15	15000.0	122.98	333.74	171.37	
1	MSFT	Microsoft	Technology	0.15	15000.0	232.95	393.82	69.06	
2	NVDA	NVIDIA	Technology	0.10	10000.0	14.28	202.81	1319.91	
3	AMZN	Amazon	Consumer Discretionary	0.10	10000.0	85.82	247.23	188.08	
4	JPM	JPMorgan Chase	Financial Services	0.15	15000.0	123.74	341.10	175.67	
5	SPY	SPDR S&P 500 ETF	ETF	0.15	15000.0	364.13	743.29	104.13	
6	GLD	SPDR Gold Shares	Commodity	0.10	10000.0	171.06	368.41	115.37	
7	BND	Vanguard Total Bond Market ETF	Bond ETF	0.10	10000.0	63.65	72.86	14.47	

```

1 # Step 35: Correct current value and profit/loss calculations
2
3 initial_by_ticker = (
4     asset_summary
5     .set_index("Ticker")["Initial_Investment"]
6 )
7
8 current_value_by_ticker = (
9     initial_by_ticker * (1 + asset_total_returns)
10 )
11
12 asset_summary["Current_Value"] = (
13     asset_summary["Ticker"].map(current_value_by_ticker)
14 )
15
16 asset_summary["Profit_Loss"] = (
17     asset_summary["Current_Value"]
18     - asset_summary["Initial_Investment"]
19 )
20
21 asset_summary.round(2)

```

	Ticker	Company	Sector	Weight	Initial_Investment	Starting_Price	Latest_Price	Total_Return_Percent	Annualized
0	AAPL	Apple	Technology	0.15	15000.0	122.98	333.74	171.37	
1	MSFT	Microsoft	Technology	0.15	15000.0	232.95	393.82	69.06	
2	NVDA	NVIDIA	Technology	0.10	10000.0	14.28	202.81	1319.91	
3	AMZN	Amazon	Consumer Discretionary	0.10	10000.0	85.82	247.23	188.08	
4	JPM	JPMorgan Chase	Financial Services	0.15	15000.0	123.74	341.10	175.67	
5	SPY	SPDR S&P 500 ETF	ETF	0.15	15000.0	364.13	743.29	104.13	
6	GLD	SPDR Gold Shares	Commodity	0.10	10000.0	171.06	368.41	115.37	
7	BND	Vanguard Total Bond Market ETF	Bond ETF	0.10	10000.0	63.65	72.86	14.47	

```

1 # Step 36: Classify each asset by volatility
2
3 def classify_risk(volatility):

```

```

4     if volatility < 15:
5         return "Low"
6     elif volatility < 25:
7         return "Medium"
8     else:
9         return "High"
10
11 asset_summary["Risk_Level"] = (
12     asset_summary["Annualized_Volatility_Percent"]
13     .apply(classify_risk)
14 )
15
16 asset_summary[
17     [
18         "Ticker",
19         "Annualized_Volatility_Percent",
20         "Risk_Level"
21     ]
22 ]

```

	Ticker	Annualized_Volatility_Percent	Risk_Level	
0	AAPL	25.730725	High	
1	MSFT	25.063707	High	
2	NVDA	48.610446	High	
3	AMZN	31.780409	High	
4	JPM	23.237250	Medium	
5	SPY	15.124975	Medium	
6	GLD	19.834710	Medium	
7	BND	5.601964	Low	

```

1 # Step 37: Rank assets by total return
2
3 asset_summary["Performance_Rank"] = (
4     asset_summary["Total_Return_Percent"]
5     .rank(
6         ascending=False,
7         method="dense"
8     )
9     .astype(int)
10 )
11
12 asset_summary = asset_summary.sort_values(
13     "Performance_Rank"
14 ).reset_index(drop=True)
15
16 asset_summary.round(2)

```

	Ticker	Company	Sector	Weight	Initial_Investment	Starting_Price	Latest_Price	Total_Return_Percent	Annualized
0	NVDA	NVIDIA	Technology	0.10	10000.0	14.28	202.81	1319.91	
1	AMZN	Amazon	Consumer Discretionary	0.10	10000.0	85.82	247.23	188.08	
2	JPM	JPMorgan Chase	Financial Services	0.15	15000.0	123.74	341.10	175.67	
3	AAPL	Apple	Technology	0.15	15000.0	122.98	333.74	171.37	
4	GLD	SPDR Gold Shares	Commodity	0.10	10000.0	171.06	368.41	115.37	
5	SPY	SPDR S&P 500 ETF	ETF	0.15	15000.0	364.13	743.29	104.13	
6	MSFT	Microsoft	Technology	0.15	15000.0	232.95	393.82	69.06	
7	BND	Vanguard Total Bond Market ETF	Bond ETF	0.10	10000.0	63.65	72.86	14.47	

```

1 # Step 38: Save the completed asset summary
2
3 asset_summary.to_csv(
4     "asset_summary.csv",
5     index=False
6 )
7
8 print("asset_summary.csv saved successfully.")
9 print("Missing values:", asset_summary.isnull().sum().sum())
10
11 asset_summary.round(2)

```

asset_summary.csv saved successfully.
Missing values: 0

	Ticker	Company	Sector	Weight	Initial_Investment	Starting_Price	Latest_Price	Total_Return_Percent	Annualized
0	NVDA	NVIDIA	Technology	0.10	10000.0	14.28	202.81	1319.91	
1	AMZN	Amazon	Consumer Discretionary	0.10	10000.0	85.82	247.23	188.08	
2	JPM	JPMorgan Chase	Financial Services	0.15	15000.0	123.74	341.10	175.67	
3	AAPL	Apple	Technology	0.15	15000.0	122.98	333.74	171.37	
4	GLD	SPDR Gold Shares	Commodity	0.10	10000.0	171.06	368.41	115.37	
5	SPY	SPDR S&P 500 ETF	ETF	0.15	15000.0	364.13	743.29	104.13	
6	MSFT	Microsoft	Technology	0.15	15000.0	232.95	393.82	69.06	
7	BND	Vanguard Total Bond Market ETF	Bond ETF	0.10	10000.0	63.65	72.86	14.47	

```

1 best_asset = asset_summary.sort_values(
2     "Total_Return_Percent",
3     ascending=False
4 ).iloc[0]
5
6 worst_asset = asset_summary.sort_values(
7     "Total_Return_Percent"
8 ).iloc[0]
9
10 highest_risk_asset = asset_summary.sort_values(
11     "Annualized_Volatility_Percent",
12     ascending=False
13 ).iloc[0]
14
15 best_sharpe_asset = asset_summary.sort_values(
16     "Sharpe_Ratio",
17     ascending=False
18 ).iloc[0]
19
20 print("ASSET PERFORMANCE SUMMARY")
21 print("-" * 45)
22
23 print(
24     f"Best performing asset: "
25     f"{best_asset['Ticker']} "
26     f"({best_asset['Total_Return_Percent']:.2f}%"
27 )
28
29 print(
30     f"Worst performing asset: "
31     f"{worst_asset['Ticker']} "
32     f"({worst_asset['Total_Return_Percent']:.2f}%"
33 )
34
35 print(
36     f"Highest-risk asset: "
37     f"{highest_risk_asset['Ticker']} "
38     f"({highest_risk_asset['Annualized_Volatility_Percent']:.2f}%"
39 )

```

```

40
41 print(
42     f"Best risk-adjusted asset: "
43     f"{best_sharpe_asset['Ticker']} "
44     f"(Sharpe Ratio: {best_sharpe_asset['Sharpe_Ratio']:.2f})"
45 )

```

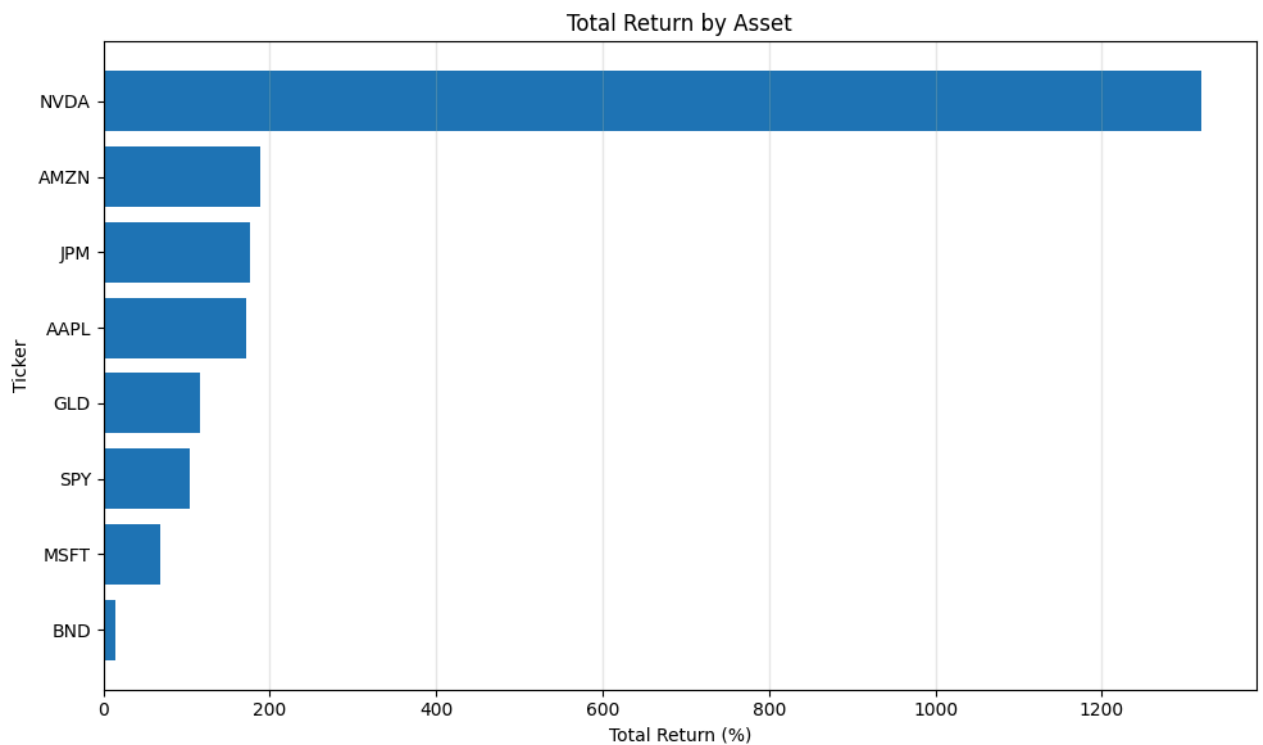
ASSET PERFORMANCE SUMMARY

Best performing asset: NVDA (1319.91%)
Worst performing asset: BND (14.47%)
Highest-risk asset: NVDA (48.61%)
Best risk-adjusted asset: NVDA (Sharpe Ratio: 2.77)

```

1 import matplotlib.pyplot as plt
2
3 chart_data = asset_summary.sort_values(
4     "Total_Return_Percent",
5     ascending=True
6 )
7
8 plt.figure(figsize=(10, 6))
9
10 plt.barh(
11     chart_data["Ticker"],
12     chart_data["Total_Return_Percent"]
13 )
14
15 plt.title("Total Return by Asset")
16 plt.xlabel("Total Return (%)")
17 plt.ylabel("Ticker")
18 plt.grid(axis="x", alpha=0.3)
19 plt.tight_layout()
20
21 plt.savefig(
22     "asset_total_returns.png",
23     dpi=300,
24     bbox_inches="tight"
25 )
26
27 plt.show()

```



```

1 allocation_data = asset_summary.sort_values(
2     "Weight",
3     ascending=False
4 )

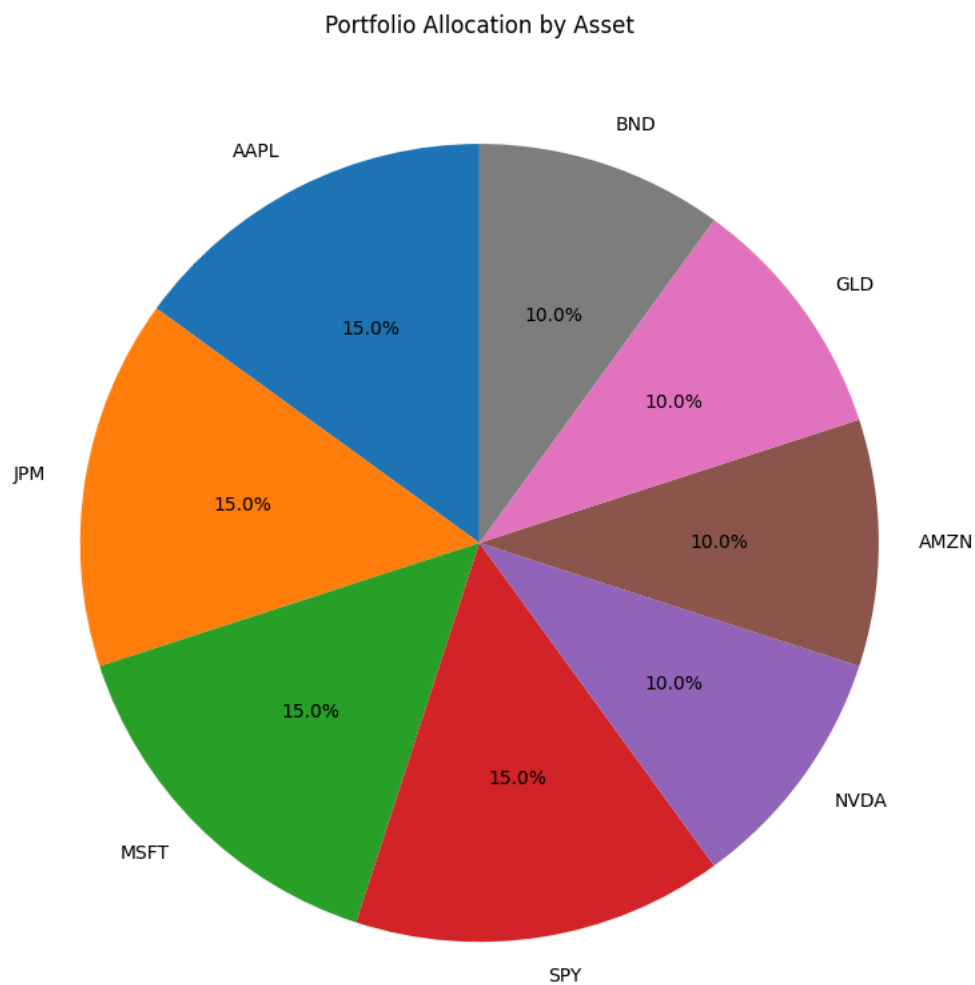
```



```

5
6 plt.figure(figsize=(8, 8))
7
8 plt.pie(
9     allocation_data["Weight"],
10    labels=allocation_data["Ticker"],
11    autopct="%1.1f%%",
12    startangle=90
13 )
14
15 plt.title("Portfolio Allocation by Asset")
16 plt.tight_layout()
17
18 plt.savefig(
19     "portfolio_allocation.png",
20     dpi=300,
21     bbox_inches="tight"
22 )
23
24 plt.show()

```



```

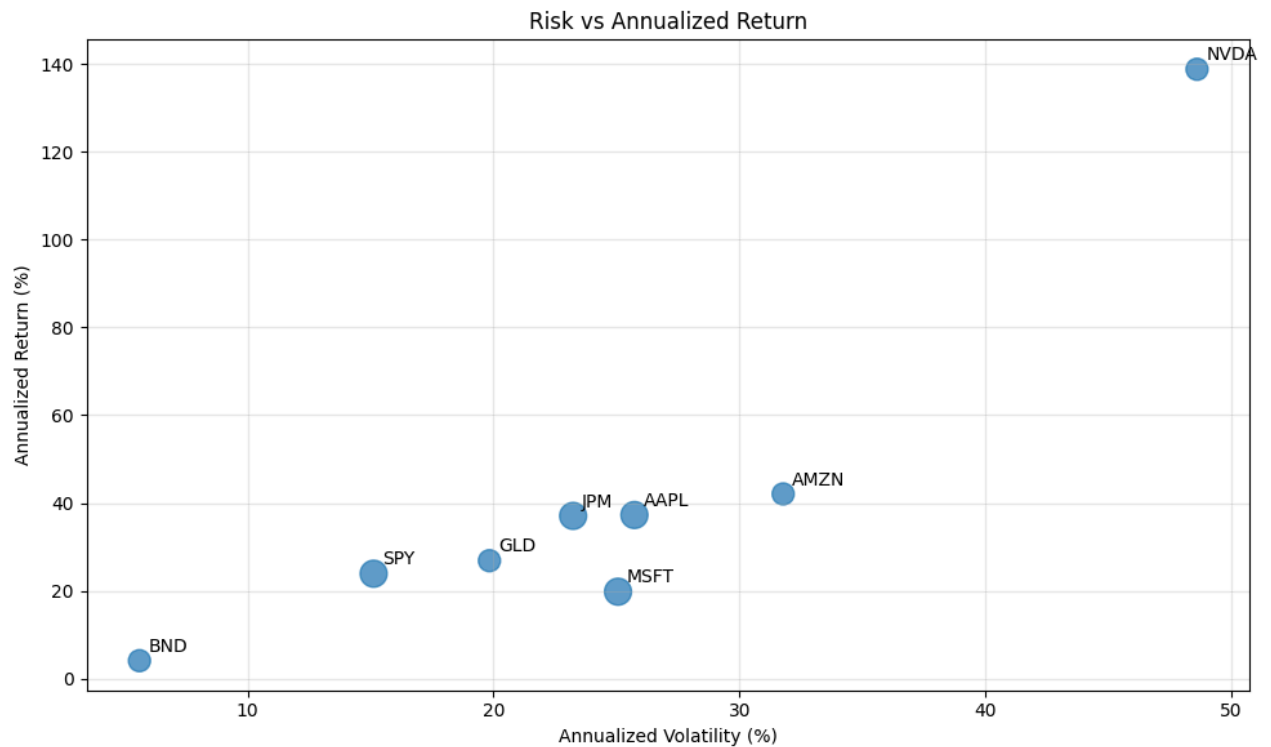
1 plt.figure(figsize=(10, 6))
2
3 plt.scatter(
4     asset_summary["Annualized_Volatility_Percent"],
5     asset_summary["Annualized_Return_Percent"],
6     s=asset_summary["Weight"] * 1500,
7     alpha=0.7
8 )
9
10 for _, row in asset_summary.iterrows():
11     plt.annotate(
12         row["Ticker"],
13         (
14             row["Annualized_Volatility_Percent"],
15             row["Annualized_Return_Percent"]
16         ),
17         xytext=(5, 5),

```

```

18     textcoords="offset points"
19 )
20
21 plt.title("Risk vs Annualized Return")
22 plt.xlabel("Annualized Volatility (%)")
23 plt.ylabel("Annualized Return (%)")
24 plt.grid(alpha=0.3)
25 plt.tight_layout()
26
27 plt.savefig(
28     "risk_vs_return.png",
29     dpi=300,
30     bbox_inches="tight"
31 )
32
33 plt.show()

```



```

1 # Recreate portfolio performance metrics
2
3 weights = asset_summary.set_index("Ticker")["Weight"]
4
5 portfolio_daily_return = daily_returns.dot(weights)
6
7 portfolio_growth = (
8     (1 + portfolio_daily_return).cumprod()
9 ) * initial_investment
10
11 current_portfolio_value = portfolio_growth.iloc[-1]
12
13 total_profit_loss = (
14     current_portfolio_value - initial_investment
15 )
16
17 total_return_percent = (
18     total_profit_loss / initial_investment
19 ) * 100
20
21 trading_days = 252
22
23 annualized_return = (
24     (1 + portfolio_daily_return.mean()) ** trading_days
25 ) - 1
26
27 annualized_volatility = (
28     portfolio_daily_return.std() * np.sqrt(trading_days)
29 )
30

```

```

31 risk_free_rate = 0.04
32
33 sharpe_ratio = (
34     annualized_return - risk_free_rate
35 ) / annualized_volatility
36
37 print(f"Sharpe Ratio: {sharpe_ratio}")

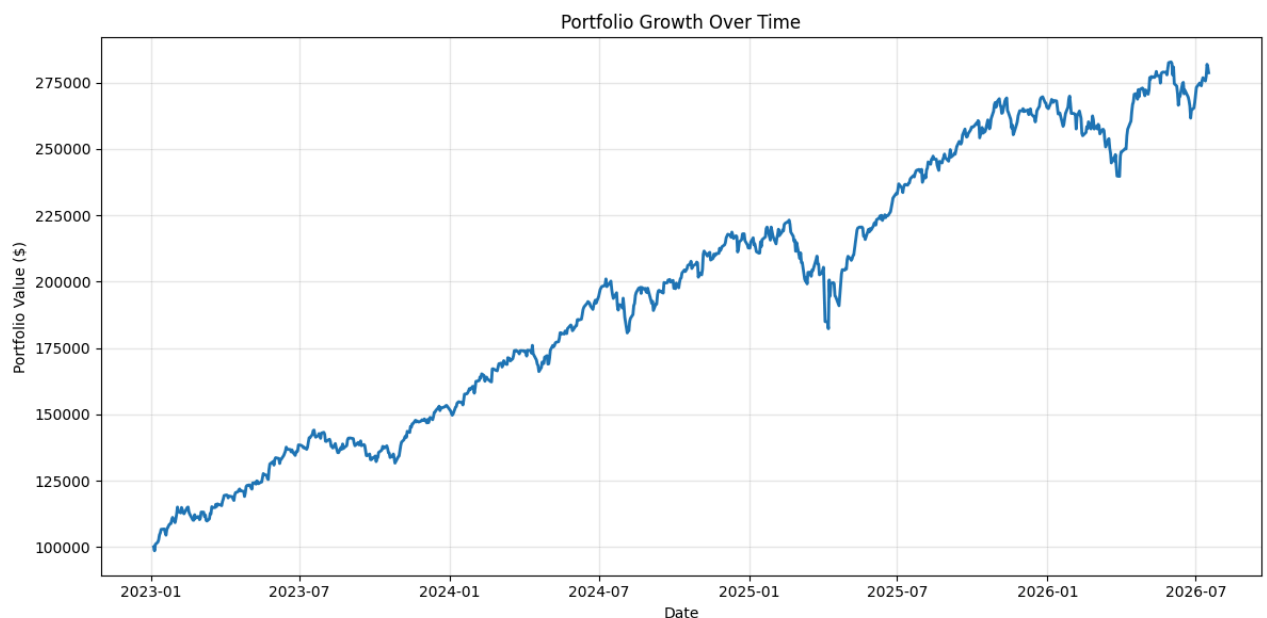
```

Portfolio metrics calculated successfully.

```

1 plt.figure(figsize=(12,6))
2
3 plt.plot(
4     portfolio_growth.index,
5     portfolio_growth.values,
6     linewidth=2
7 )
8
9 plt.title("Portfolio Growth Over Time")
10 plt.xlabel("Date")
11 plt.ylabel("Portfolio Value ($)")
12 plt.grid(alpha=0.3)
13
14 plt.tight_layout()
15 plt.show()

```



```

1 portfolio_kpis = pd.DataFrame({
2     "Metric":[
3         "Initial Investment",
4         "Current Portfolio Value",
5         "Profit/Loss",
6         "Total Return (%)",
7         "Annualized Return (%)",
8         "Annualized Volatility (%)",
9         "Sharpe Ratio"
10    ],
11    "Value":[
12        initial_investment,
13        current_portfolio_value,
14        total_profit_loss,
15        total_return_percent,
16        annualized_return*100,
17        annualized_volatility*100,
18        sharpe_ratio
19    ]
20 })

```

	Metric	Value	
0	Initial Investment	100000.000000	 
1	Current Portfolio Value	278658.373592	
2	Profit/Loss	178658.373592	
3	Total Return (%)	178.658374	
4	Annualized Return (%)	35.542225	
5	Annualized Volatility (%)	15.950565	
6	Sharpe Ratio	1.977499	

Next steps: [Generate code with portfolio_kpis](#) [New interactive sheet](#)

```

1 # Save final datasets
2
3 asset_summary.to_csv(
4     "asset_summary.csv",
5     index=False
6 )
7
8 daily_returns.to_csv(
9     "daily_returns.csv",
10    index=True
11 )
12
13 portfolio_growth_df = pd.DataFrame({
14     "Date": portfolio_growth.index,
15     "Portfolio_Value": portfolio_growth.values
16 })
17
18 portfolio_growth_df.to_csv(
19     "portfolio_growth.csv",
20     index=False
21 )
22
23 portfolio_kpis.to_csv(
24     "portfolio_kpis.csv",
25     index=False
26 )
27
28 closing_prices.to_csv(
29     "closing_prices.csv",
30     index=True
31 )
32
33 print("All CSV files saved successfully.")

```

All CSV files saved successfully.

```

1 import sqlite3
2
3 connection = sqlite3.connect(
4     "investment_portfolio.db"
5 )
6
7 asset_summary.to_sql(
8     "asset_summary",
9     connection,
10    if_exists="replace",
11    index=False
12 )
13
14 daily_returns.reset_index().to_sql(
15     "daily_returns",
16     connection,
17     if_exists="replace",
18     index=False
19 )
20
21 portfolio_growth_df.to_sql(
22     "portfolio_growth",
23     connection,
24     if_exists="replace",

```

```

25     index=False
26 )
27
28 portfolio_kpis.to_sql(
29     "portfolio_kpis",
30     connection,
31     if_exists="replace",
32     index=False
33 )
34
35 closing_prices.reset_index().to_sql(
36     "closing_prices",
37     connection,
38     if_exists="replace",
39     index=False
40 )
41
42 connection.close()
43

```

SQLite database created successfully.

```

1 import os
2
3 files = [
4     "asset_summary.csv",
5     "daily_returns.csv",
6     "portfolio_growth.csv",
7     "portfolio_kpis.csv",
8     "closing_prices.csv",
9     "investment_portfolio.db"
10 ]
11
12 for file in files:
13     print(
14         file,
15         "-",
16         "Created" if os.path.exists(file) else "Missing"
17     )

```

asset_summary.csv - Created
daily_returns.csv - Created
portfolio_growth.csv - Created
portfolio_kpis.csv - Created
closing_prices.csv - Created
investment_portfolio.db - Created

```

1 import shutil
2 import os
3
4 project_folder = "Investment_Portfolio_Analytics"
5
6 os.makedirs(project_folder, exist_ok=True)
7
8 files_to_copy = [
9     "asset_summary.csv",
10    "daily_returns.csv",
11    "portfolio_growth.csv",
12    "portfolio_kpis.csv",
13    "closing_prices.csv",
14    "investment_portfolio.db",
15    "asset_total_returns.png",
16    "portfolio_allocation.png",
17    "risk_vs_return.png"
18 ]
19
20 for file in files_to_copy:
21     if os.path.exists(file):
22         shutil.copy(file, project_folder)
23
24 shutil.make_archive(
25     "Investment_Portfolio_Analytics",
26     "zip",
27     project_folder
28 )
29
30 print("ZIP file created successfully.")

```

ZIP file created successfully.

```
1 from google.colab import files  
?
```